# ASC Student Supercomputer Challenge 2025 Preliminary Contest Proposal

Fuzhou University
Team: FZUASC Team Advisor: Xiangwen Liao Date: 2025.1.10
Members: Zhixuan Zhang, Zishuo Bao, Xuanfen Zhang, Zhenye Xie, Jiaqi Rong

***Abstract*—In the preliminary contest, we harness the cluster of the Fujian Super Computing Center to manage the challenges. As for HPC system design, we use Inspur NF5280M6 cluster to provide two types of schemes to adapt the computing need of optimization for AlphaFold inference challenge and RNA Modification Site Detection and Performance Optimization Challenge. According to our actual cluster, we completed the HPL and HPCG tests to estimate the performance. As for the AlphaFold inference challenge, we employed a variety of strategy combinations to speed up the inference process while meeting the requirements of the question and proposed some potentially superior alternatives. For the RNA challenge, we redesign the program structure of a bottleneck program in workflow and implement it in a efficent way, reach a $6\times$ improvement in bottleneck.**

***Index Terms*—HPC, AI, Alphafold3, HPL**

## I. Supercomputing-related hardware and software platforms

The Fujian Supercomputing Center at Fuzhou University is a major special project constructed by the public infrastructure service platform "Digital Fujian". The project was approved by the Fujian Development and Reform Commission in January 2008, of which the total investment budget is 16.41 million yuan. The high-performance computing public service platform with complete functions, high efficiency, and complete system takes Fuzhou University as the main body. Also, the Fujian Supercomputing Center combines with the computing resources of the high-performance computing center which has been built by Xiamen University.

The second phase of the Fujian Supercomputing Center has been formally put into use after a period of trial operation. Benefiting from these improvements, one of the major features of the Fujian Supercomputing Center is high-performance parallel computing with 211 nodes which achieves 800 trillion times per second of calculating speed. The second phase of Fujian Supercomputing Center would serve in the fields of meteorology, environmental protection, marine, medical, automobile, and so on. It also supports technology cooperation for government departments, enterprises and institutions, universities, and research institutions in Fujian. The Fujian Supercomputing Center would strive to become the national firstclass high-performance computing public resources service center, high-performance computing technical support center, and high performance computing value-added services center.

TABLE I: HARDWARE RESOURCES

| Name | Details |
|---|---|
| System peak | 13.58 Tflops |
| Computing nodes | 290 4-way Intel Xeon Processors |
| Storage node | 12 4-way Intel Xeon Processors |
| Acess node | 4 4-way Intel Xeon Processors |
| CPU | Intel Xeon E5440, 2.83GHz $\times$ 1224 |
| Total system memory | 2448 GB |
| Total disk capacity | 20 TB |
| Architecture | Fluster, InfiBand |
| Operating System | CentOS 5.2 |

TABLE II: SOFTWARE RESOURCES

| Name | Details |
|---|---|
| Languages | C, C++, Fortran77/90/95, Java |
| Compilers | GCC, Javac, tcl/tk, Perl |
| Math Libraries | GOTOBlast, Openblas, Intel MKL |
| Parallel Environments | Openmpi, Ofed, Ray |
| Tools Softwares | OpenLDAP2.3, Python3.12 |
| System Management | Sun Grid Engine 6.1 |

### A. Hardware resources and software resources

The hardware resources is shown in **Table I** ,and the software resources is shown in **Table II**.

### B. Supercomputing-related courses, trainings and interest groups

- Introduction of high-performance computing
- High performance computing program design
- Parallel computing

### C. Supercomputing-related research and applications

- New materials, new energy
- Automobile and shipbuilding
- Civil Engineering
- Climate Meteorology and Environmental Sciences
- Creature Medicine
- Physical Chemistry
- Aerospace

### D. A detailed description of the key achievements on supercomputing research

In 2014, Fuzhou University won the "outstanding contribution" unit of China Education and Research Computer Network

In 2015 SCI "FLUID PHASE EQUILIBRIA" published in Fujian Province supercomputing center computing platform user results

## II. Team Introduction

### A. The building process of FZUASC Team

FZUASC is a team composed of different professional compositions and different grades of students which was founded in October 2017. As achieving 30th place in ASC17, and winning first prize in ASC18, ASC19, ASC20-21, ASC22-23 and ASC24, we sincerely expect another step forward under the guidance of tutors. Adhering to the concept that "passion, innovation, and hard work" to participate in this competition, the FZUASC team will challenge these missions to achieve self-realization. We come for the dream and we strive to move forward.

### B. Team members

**Advisor: Xiangwen Liao**

Xiangwen Liao born in 1980, Professor, CCF senior member. He received his PhD degree in computer architecture from the Institute of Computer Sciences, Chinese Academy of Sciences. His research interests include information retrieval, social media process, natural language processing, and high-performance computing.

**Captain: Zhixuan Zhang**

A junior student from the School of Computer Science and Big Data, with a keen interest in computer applications, aspires to apply their computer knowledge to solve practical problems in the field of natural sciences.

The blessing: I hope that our team will achieve success in the competition and wish for everyone's aspirations to come true. We also hope that the ASC competition will continue to thrive and improve.



**Member: Zishuo Bao**

A junior student from Maynooth International Engineering College, Fuzhou University. Majoring in software engineering, mainly responsible for the RNA $m^5C$ part of ASC25.

The blessing: Best Wishes!



**Member: Xuanfen Zhang**

A junior student from the School of Computer Science and Big Data.He has a strong interest in the field of computer and artificial intelligence, and he is mainly responsible for the Alphafold3 inference acceleration section in this competition.

The blessing: Hope to learn more from this competition.



**Member: Zhengye Xie**

A junior student from Maynooth International Engineering College, Fuzhou University. Majoring in software engineering, mainly responsible for the Alphafold3 part of ASC25.

The blessing: Win, Win, Win! Six hundred and sixty six!



**Member: Jiaqi Rong**

A junior student in the School of Computer and Big Data. Actively participate in school laboratories and competitions. This competition is mainly responsible for Alphafold3 inference acceleration tasks and HPL&HPCG.

The blessings: The team can achieve good results.



### C. Tearm slogan and logo

Our slogan is "Everything Can Be Computed, Infinite Possibilities Await" As for our logo, as you can see in the picture, our logo consists of a design and our team name. The upper part consists of three letters: "A", "S" and "C". Also, it shows the deep connotation of our team through the red color and the design is rich in the times, besides our team's strong dedication to ASC.

FZU ASC

Fig. 1: FZU ASC Team's Logo



Fig. 2: FZU ASC Team's Photo

TABLE III: CPU NODE CONFIGURATION

| Item | Name | Configuration |
|---|---|---|
| Server | Inspur NF5290M6 | CPU: Intel Xeon Platinum 8468×2, 2.10GHz, 48 cores |
| | | Memory: 32 GB×12, DDR4, 2933 Mhz |
| | | Hard disk: 480G SSD SATA×1 |
| | | Power consumption estimation: 8468 TDP 350W, memory 7.5W, hard disk 10W |
| HCA Card | EDR | InfiniBand Mellanox ConnectX®-5 HCA card, single port, QSFP, EDR IB |
| | | Power consumption estimation: 9W |
| Switch | GbE switch | 10/100/1000Mb/s, 24 ports Ethernet switch |
| | | Power consumption estimation: 30W |
| | EDR-IB switch | Switch-IB™ EDR InfiniBand, 36 QSFP port |
| | | Power consumption estimation: 130W |
| Cabel | Gigabit CAT6 | CAT copper cable, 3m |
| | InfiniBand cable | InfiniBand EDR copper cable, QSFP port, cooperating with the InfiniBand switch for use |

## III. HPC System Design

### A. Overview

For the specific application of this preliminary contest, we have completed the test based on the heterogeneous cluster with GPUs and CPU-only cluster, so we provide two types of design for HPC systems as below.

### B. CPU node

The configuration of each CPU node is shown in **Table III**.

### C. Accelerators scheme

We propose a solution for GPUs. We are pleased to have a chance to test the NVIDIA A100 with the help of Fujian Supercomputing Center. We learn from Preliminary Round Notification that each Inspur NF5280M6 server supports four GPUs at most. We choose the NVIDIA A100 with an estimated power consumption of 400W each. Based on this, we decided to install four A100 GPUs for 1 node so that we can limit the power consumption. We use CPU-GPU heterogeneous acceleration calculations in these nodes. The configuration is shown in **Table IV**.

### D. System evaluation

Our system has the advantages as following.

1) According to the computing resources of RNA Chanllenge needs, adequate CPUs are necessary. In addition, GPUs are supposed to be needed to accelerate the AlphaFold3 part.
2) GPUs make great contributions in accelerating.
3) The structure of different nodes is the same which promises the load balancing among nodes to make sure to get a better performance.

TABLE IV: GPU NODE CONFIGURATION

| Item | Name | Configuration |
|---|---|---|
| Server | Inspur NF5290M6 | CPU: Intel Xeon Platinum 8468×2, 2.10GHz, 48 cores |
| | | Memory: 32 GB×12, DDR4, 2933 Mhz |
| | | Hard disk: 480G SSD SATA×1 |
| | | Power consumption estimation: 8468 TDP 350W, memory 7.5W, hard disk 10W |
| HCA Card | EDR | InfiniBand Mellanox ConnectX®-5 HCA card, single port, QSFP, EDR IB |
| | | Power consumption estimation: 9W |
| Switch | GbE switch | 10/100/1000Mb/s, 24 ports Ethernet switch |
| | | Power consumption estimation: 30W |
| | EDR-IB switch | Switch-IB™ EDR InfiniBand, 36 QSFP port |
| | | Power consumption estimation: 130W |
| Accelerator GPU | | NVIDIA A100 80GB × 4 |
| | | Power consumption per card: 400W |

However the cluster we use in the preliminary contest is different from the cluster architecture we designed above. So we can only make a theoretical analysis, such as the control of power consumption. The Theoretical performance of our design is shown in Table V

TABLE V: THEORETICAL ANALYSIS FOR PERFORMANCE

| Mode | Node | GFLOPS |
|---|---|---|
| CPU | 3 | 77414.4 |
| CPU+GPU | 1 | 105676.8 |

## IV. HPL & HPCG

### A. Overview

In this part, we utilize two types of software, HPL and HPCG, to assess the performance of our clusters with the aim of identifying the highest level of performance and understanding the relationship between performance and power consumption. We will provide details on the software environment, testing methodology, performance optimization techniques, performance estimation, problem analysis, and proposed solutions, among other aspects.

### B. Introduction of HPL & HPCG

HPL is a software package that solves a random dense linear system in double precision (64-bits) arithmetic on distributed-memory computers [1].

HPCG is a software package that performs a fixed number of multigrid preconditioned (using a symmetric Gauss-Seidel smoother) conjugate gradient (PCG) iterations using double precision (64 bit) floating point values.

### C. Hardware and software environment

The platform we use to test and optimize is a GPU heterogeneous platform. The hardware of our platform is shown in **Table VI** and the software is shown in **Table VII**. We have two nodes with identical configurations, and they are interconnected using an InfiniBand (IB) network.

TABLE VI: THE CONFIGURATION OF CLUSTER

| Name | Details |
|---|---|
| CPU | Intel(R) Xeon(R) Gold 6132 CPU @ 2.60GHz |
| GPU | 2×NVIDIA A800 80GB PCIe |

TABLE VII: SOFTWARE ENVIRONMENT

| Classification | Description | Version |
|---|---|---|
| Operating system | Manjaro | 25.0.0 |
| Compiler | Intel Compilers | 2024.09 |
| MKL | Intel MKL | 2024.0 |
| MPI | mpich | 4.3.0 |

### D. HPL Benckmark

We will use HPL-2.3 to test CPU performance and performance on both CPU and GPU via Nvidia HPL Benchmark Docker. The highest performance is $\mathbf{2.969 \times 10^4}$ GFLOPS.

The algorithms used by HPL can be summarized with the following keywords:

- Two-dimensional block cyclic data distribution;
- LU decomposition of the right variant with row partial pivot element;
- Multiple foresight depths;
- Recursive panel decomposition, combining principal meta search and column broadcast;
- Multiple virtual panel broadcast topologies;
- Switching and broadcasting algorithm with reduced bandwidth;
- Forward substitution with depth 1.

Therefore, the power consumption of this algorithm will be higher than that of ordinary software. We conducted some tests to find out the relationship between performance and power consumption in order to achieve the highest performance within the limited power consumption.

a) *Compiling and running HPL Benchmark on CPU:*

Since our servers use intel Intel(R) Xeon(R) Gold 6132 CPU@2.60GH, we use `Make.Linux_Intel64` to compile. We set the compile flags and linker flags like **Fig. 3**.

```
CC          = mpiicc
CCNOOPT     = $(HPL_DEFS)
OMP_DEFS    = -openmp
CCFLAGS     = $(HPL_DEFS) -O3 -w -ansi-alias -z relro -Z now -fiopenmp

LINKER      = $(CC)
LINKFLAGS   = $(CCFLAGS) $(OMP_DEFS) -mt_mpi
```

Fig. 3: The Configure of HPL

Then use the following commands

```
cp ./setup/Make.Linux_Intel64 Make.test
make arch=test
```

After the success of compiling, `HPL.dat` and `xhpl` are generated in the bin/test director.

b) *Performance optimization methods:*

We use the following methods to optimize performance of our cluster in HPL test.

- Use the optimal library, like OpenBLAS;
- Turn on compiler optimization like `-O3`;
- Optimize the parameters in `HPL.dat`.

We will mainly introduce the third method.

c) *Tuning Method:*

In `HPL.dat`, we mainly focus on 3 parameters: NB, N and P×Q. We do the test as the following steps.

Step 1: pick a small value of N;

Step 2: find the best value of NB;

Step 3: find the best value of P×Q;

Step 4: find the best value of N.

d) *Block size NB:*

The block size NB is the computational granularity. NB times 8 must be a multiple of the cache line, generally less than 256. According to the test experience, the size choice of NB is also related to communication, matrix size, network, processor speed and other factors.

Typically, a single node or a single CPU test can get some good NB values, but as the system size increases and the problem size grows, the performance of some NB values can become worse. Therefore, it is best to choose about 3 NB values that perform well in small-scale tests and then validate them in large-scale tests.

When NB is too small, the traffic is greater than the computational optimization, resulting in a significant drop in speed. When NB is too large, it affects load balancing (so the effect of NB on a single node is not as significant).

So we use `lscpu` to know the cache line of CPU is 64. Because NB $\times$ 8 must be a multiple of the cache line, we pick a series of typical values:

```
96, 112, 128, 160, 192, 240, 256, 320,
384, 480, 512, 640, 768, 960, 1024
```

**Fig. 4** shows the relationship between GFLOPS and NB, where NB equals to 240, the performance reach the top.

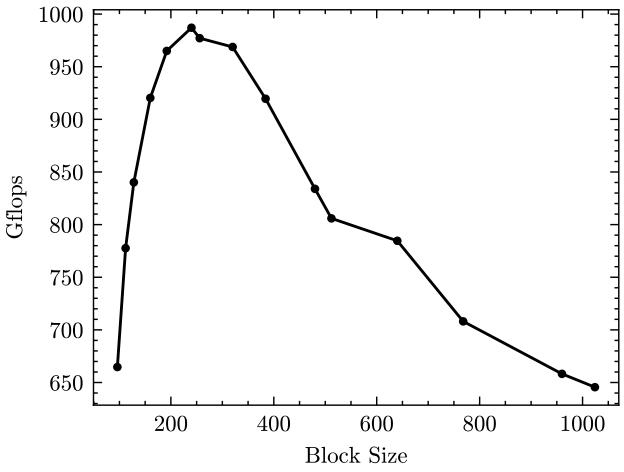



Fig. 4: Relationship Between Gflops and block size, $N = 30000, P \times Q = 2 \times 14$

e) *2D Processor Grid (P×Q):*

Make $P \times Q$ equals to the number of the processors is a hard rule when performing HPL, while $P \leq Q$ is a test experience value.

Generally speaking, the value of P would be as small as possible, because column communication traffic (communication times and communication data) is much larger than horizontal communication traffic.

$P = 2^n$, which is the best choice of P, make P equals to the powers of 2. In HPL, the column communication L decomposed using Binary Exchange. Then when P equals to the powers of 2, it would get the best performance. In addition, in the broadcast of U, Long and Binary Exchange would bring best performance as well. When the number of processes is a square number,it may bring better performance. The best $P \times Q$ is $8 \times 14$ which can be infer from the **Fig. 5**.

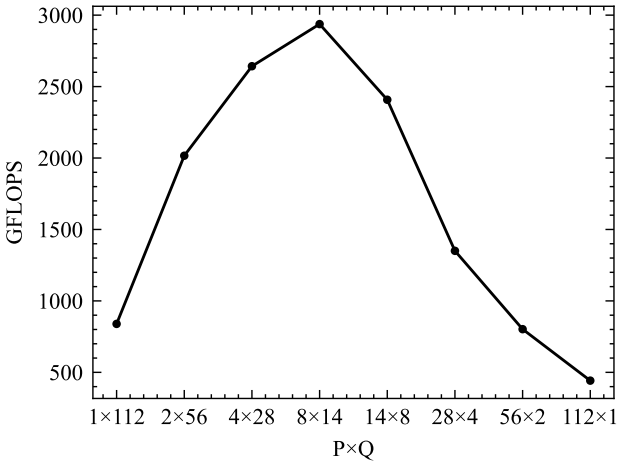



Fig. 5: Relationship Between Gflops and $P \times Q$, $N = 30000, \text{NB} = 240$

f) *Matrix size N:*

The larger size of matrix, the larger proportion the effective calculation occupies and then higher efficiency of the system floating-point processing performs. But at the same time, the increasing size of matrix will lead to an increasing memory consumption. Once there is little the system actual memory space, available cache and performance will be greatly reduced.

Therefore, as far as the general system, we should try the best not only to increase the size of the matrix N, but also to ensure that no system cache would be used. Generally speaking, about 80% matrix A taking up the total memory of the system is the best, in other words, $N \times N \times 8 \approx$ memory size $\times$ 80%, while keep N generally can be divisible by NB $\times$ LCM$(P, Q)$, among which LCM represents the minimum divisor of P and Q. We got the final result in **Fig. 6**.

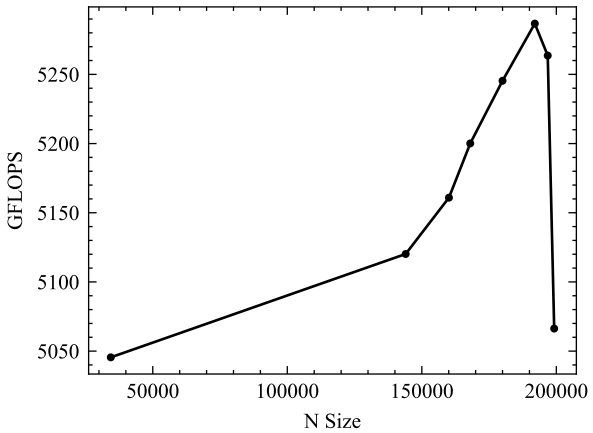



Fig. 6: RelationShip Between Gflops and N
NB $= 240, P \times Q = 8 \times 14$

g) *Run HPL Benchmark on GPU platform:*

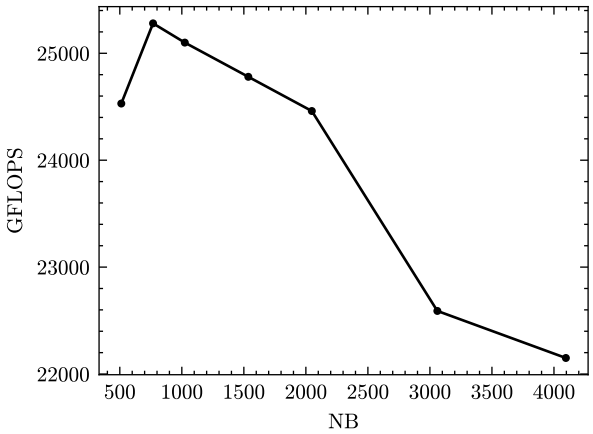



Fig. 7: Relationship between GFLOPS and NB
$N = 90000, P \times Q = 2 \times 1$

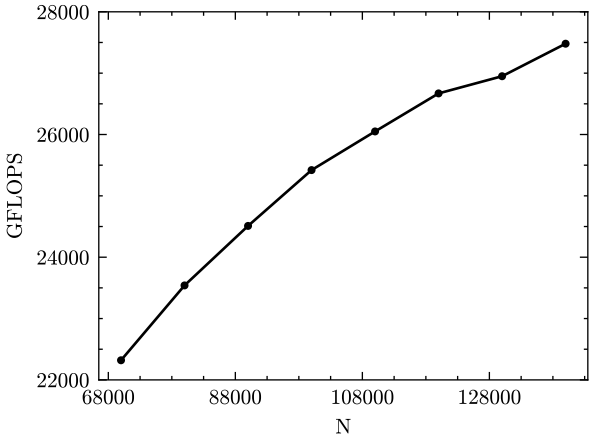



Fig. 8: RelationShip Between Gflops and N
NB $= 768, P \times Q = 2 \times 1$

We test GPU through hpl based on CUDA 12. The highest performance is 29 TFLOPS. We use the same method to optimize the parameters in hpl based on GPU platform. Record the data as shown in **Fig. 8** and **Fig. 7**. According to result, we found that the best NB is 768 while the best $P \times Q = 1 \times 2$ in GPU nodes.

h) *Performance estimation:*

For CPU, we referring to Intel's website, we find that the frequency of Intel(R) Xeon(R) Gold 6132 CPU is 2.6 GHz, we have 28 cores for per socket and have three node in all. We got the best result which shown in **Fig. 9**.

Therefore, the theoretical value is

$$2.6 \text{ GHZ} \times 32 \text{ Cores} \times 32 \text{ flop} = 2662.4 \text{ GFLOPS} \quad (1)$$

and the efficiency ratio is

$$\frac{5286.7}{4 \times 2662.4} \times 100\% = 49.6\% \quad (2)$$

For GPU, referring to NVIDIA's website, we find that the single-Precision performance of single A800 is 19.49 TFLOPS.

The best performance of hpl test on GPU platform is shown in **Fig. 10**. The efficiency ratio is

$$\frac{296900}{389800} \times 100\% = 76.2\% \quad (3)$$

*E. HPCG Benckmark*

We use HPCG 3.0 to test performance on the CPU and performance on GPU through HPCG 3.1 Binary for NVIDIA GPUs including Ampere. The highest performance is **437.503 GFLOPS**.

High performance the conjugate gradient benchmark (HPCG) is an iterative algorithm widely used, compared with the traditional HPL mainly focuses on test performance calculation in linear equations, HPCG differential equations are more complicated calculations, and can deal with those of memory for a large number of irregular access to high memory and low latency network requirements thus, more

```
================================================================================
T/V                N    NB     P     Q               Time                 Gflops
--------------------------------------------------------------------------------
WR00L2L2      192000   240     8    14             892.55             5.2867e+03
HPL_pdgesv() start time Tue Jan  21 12:33:54 2025

HPL_pdgesv() end time   Tue Jan  21 12:48:47 2025

--VVV--VVV--VVV--VVV--VVV--VVV--VVV--VVV--VVV--VVV--VVV--VVV--VVV--VVV--VVV-
Max aggregated wall time rfact . . . :               6.10
+ Max aggregated wall time pfact . . :               4.51
+ Max aggregated wall time mxswp . . :               4.31
Max aggregated wall time update  . . :             818.72
+ Max aggregated wall time laswp . . :              57.12
Max aggregated wall time up tr sv  . :               0.70
--------------------------------------------------------------------------------
||Ax-b||_oo/(eps*(||A||_oo*||x||_oo+||b||_oo)*N)=   7.56639447e-04 ...... PASSED

================================================================================
```

Fig. 9: The best performance of CPU platform

```
================================================================================
T/V                N    NB     P     Q               Time                 Gflops (   per GPU)
--------------------------------------------------------------------------------
WC0           139776   768     1     2              61.32              2.969e+04 ( 1.484e+04)

HPL_pdgesv() start time Fri Feb 14 13:17:26 2025
HPL_pdgesv() end time   Fri Feb 14 13:18:28 2025

--------------------------------------------------------------------------------
||Ax-b||_oo/(eps*(||A||_oo*||x||_oo+||b||_oo)*N)=   0.000510674677 ...... PASSED
||Ax-b||_oo  . . . . . . . . . . . . . . . . . . . = 0.0000000007754554
||A||_oo . . . . . . . . . . . . . . . . . . . . . = 35190.4728356468549464
||x||_oo . . . . . . . . . . . . . . . . . . . . . = 2.7806243634204111
||b||_oo . . . . . . . . . . . . . . . . . . . . . = 0.4999980925531585
================================================================================

Finished      1 tests with the following results:
              1 tests completed and passed residual checks,
              0 tests completed and failed residual checks,
              0 tests skipped because of illegal input values.
--------------------------------------------------------------------------------
```

Fig. 10: The best performance of GPU platform

practical. We will introduce the testing method, performance optimization methods, performance estimation, etc.)

a) *Compiling and running HPCG:*

Firstly, upload HPCG compressed package to our cluster. And then use `tar -zxvf hpcg-3.0.tar.gz` to decompress hpcg3.0, then use the following commands

```
cd hpcg-3.0/setup
cp Make.Linux Make.test
```

Then open the `Make.test` file, and modify the necessary parameters in the `Make.test` file. After modifying the environment configuration, save and compile.

```
cd build
make
```

After the success of compiling, there are `HPCG.dat` and `xhpcg` files generated in `build/bin` directory.

b) *Performance optimization methods:*

We use the following methods to optimize performance of the HPCG Test.

- use the optimal library
- turn on compiler optimization like -O3
- optmize the parameters in HPCG.dat

We will mainly introduce the third method.

c) *HPCG Benchmark in CPU:*

In problem size selection, the minimum value of the scale of the CPU problem is 24, and it must be a multiple of 8. The size of the hpcg problem that chooses CPU is different from that of GPU. The size of the problem should be larger than the size of Cache, otherwise it will lose the authenticity of the result, which means that the scale of the problem is at least larger than that of "main memory" 1/4. We choose the size of several groups of problems and set the time to be 180. The local domain dimension defines the global domain dimension by $(\mathrm{NR} \times \mathrm{Nx}) \times (\mathrm{NR} \times \mathrm{Ny}) \times (\mathrm{NR} \times \mathrm{Nz})$, where $\mathrm{Nx} = \mathrm{Ny} = \mathrm{Nz}$ and NR is the number of MPI processes. Here we set the number of MPI processes to 64.

```
Benchmark Time Summary=
Benchmark Time Summary::Optimization phase=1.78814e-07
Benchmark Time Summary::DDOT=9.83519
Benchmark Time Summary::WAXPBY=0.594517
Benchmark Time Summary::SpMV=4.11496
Benchmark Time Summary::MG=33.7819
Benchmark Time Summary::Total=48.3366
```

Fig. 11: An example of 16*16*16

From the changing trend shown in **Fig. 11**, it's easily to find that the smaller the scale of the problem is, the faster the operation will be. The reason for which is that the smaller scale allows data to be stored in Cache, accelerating data access and speeding up computation. Among the three groups of `8, 16, 32, 64, 128, 152`, the peak shape is presented, and the best result is obtained at $16 \times 16 \times 16$, **48.33 GFlops**. All tests are allocated to 64 processes, each of which contains a thread.

It can be seen that the run time of each part of the program is not significantly different from that of the GPU version, and the most occupied time is `MG`, followed by `SpMV`, `WAXPBY` and `DDOT`.

By analyzing the differences between **Fig. 12** and **Fig. 13**, we can find that in the hpcg calculated by CPU, the run time of `DDOT` is less than that of GPU; `SpMV` and `SymGS` accounting for most of the run time of programs.

The output data is shown in **Fig. 14**.

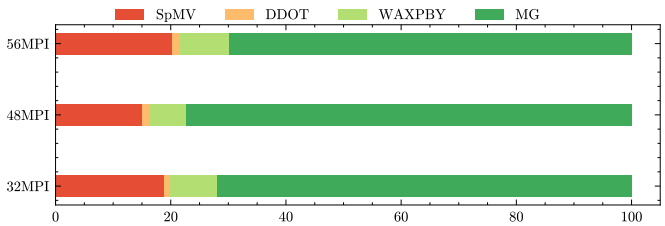



Fig. 12: Different Process Comparison

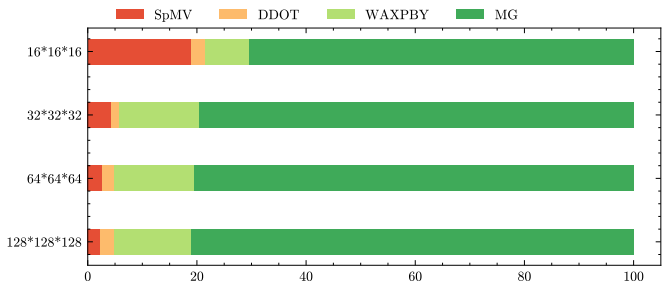



Fig. 13: Different Size Comparison

```
########## Performance Summary (times in sec) ##########=
Benchmark Time Summary=
Benchmark Time Summary::Optimization phase=1.78814e-07
Benchmark Time Summary::DDOT=9.83519
Benchmark Time Summary::WAXPBY=0.594517
Benchmark Time Summary::SpMV=4.11496
Benchmark Time Summary::MG=33.7819
Benchmark Time Summary::Total=48.3366
Floating Point Operations Summary=
Floating Point Operations Summary::Raw DDOT=4.04546e+10
Floating Point Operations Summary::Raw WAXPBY=4.04546e+10
Floating Point Operations Summary::Raw SpMV=3.55337e+11
Floating Point Operations Summary::Raw MG=1.9767e+12
Floating Point Operations Summary::Total=2.41295e+12
Floating Point Operations Summary::Total with convergence overhead=2.41295e+12
GB/s Summary=
GB/s Summary::Raw Read B/W=307.792
GB/s Summary::Raw Write B/W=71.1422
GB/s Summary::Raw Total B/W=378.934
GB/s Summary::Total with convergence and optimization phase overhead=323.093
GFLOP/s Summary=
GFLOP/s Summary::Raw DDOT=4.11325
GFLOP/s Summary::Raw WAXPBY=68.0462
GFLOP/s Summary::Raw SpMV=86.3526
GFLOP/s Summary::Raw MG=58.5137
GFLOP/s Summary::Raw Total=49.9197
GFLOP/s Summary::Total with convergence overhead=49.9197
GFLOP/s Summary::Total with convergence and optimization phase overhead=42.5633
```

Fig. 14: Summaries of parts

According to the **Fig. 15**. we also found that the performance of more processes is better than the fewer number of processes by comparison with the number of different processes in the same size.

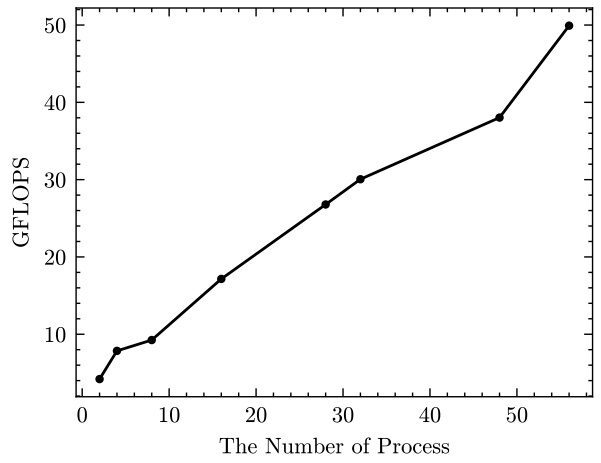



Fig. 15: Different MPI Process Comparison

As is easily known above, when the size of the matrix is $16 \times 16 \times 16$, and the number of MPI processes is 64, the best result is achieved, Gflops. The result is shown in Fig. 14 The above figures further lets us determine that under the scale of $16 \times 16 \times 16$, the 64 MPI process makes the result of the program up to the highest. When it comes to $192 \times 192 \times 192$ and bigger scale, HPCG collapses. To summarize, the size, $16 \times 16 \times 16$, and 64 MPI processes are the most suitable for the machine.

d) *HPCG Benchmark in GPU:*

HPCG benchmark input file includes 4 lines of commands with the top 2 commands do not use, the remaining two followed is the dimensions of each MPI process (such as 104 104 104), and the number of seconds in the program control part operation.

In the formal test, the duration is a minimum of 3600 seconds or more, which typically corresponds to one hour, usually 3660 seconds. The following results are given under different sizes.

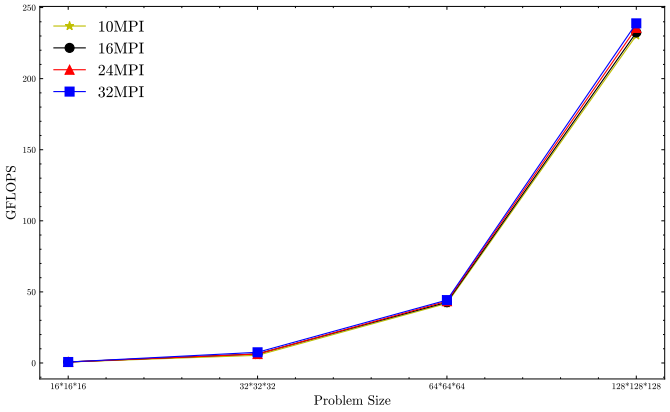



Fig. 16: Detailed data about performance

As shown in **Fig. 16**, the best running result of the problem is $256 \times 256 \times 256$, the memory of GPU is slightly more than 69G at runtime, and the kernel use rate is close to 100%.

The Memory Usage of A800 reached 84.8%. Since the memory limit, we only test 10 MPIs in the problem size of $256 \times 256 \times 256$. Within a certain range, the greater the memory usage and the more the number of MPIs, the better the GFLOPS performance will be.

The results indicate that the HPCG program run time is mainly composed of two parts and the run time of the `SymGS` (`MG` consists of `SpMV` and `SymGS` component), the main reason is that HPCG has a random access memory, due to the great random change, the program locality is very poor, and the hit rate of cache also decrease ,which means that program requires repeated access and loading data blocks, and read the data read or write operation for the sum of the most from memory, which should large than 80%, so that the program run time is mainly focused on these two parts.

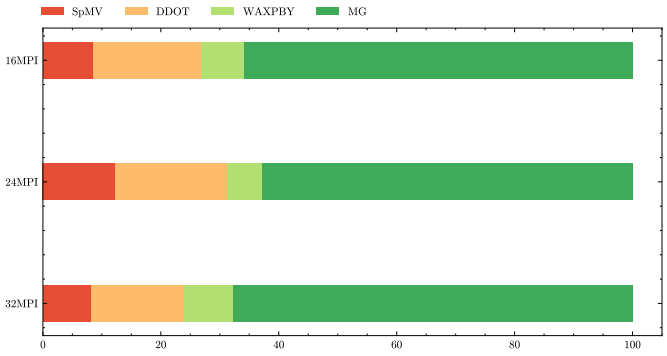



Fig. 17: Different MPI Process Comparison

The **Fig. 17** shows the $128 \times 128 \times 128$ problem size, parts of the GPU MPI number of different time-consuming operation. It can be seen from the figure, `MG` occupies the largest part in overall operation time in `SymGS` operation followed by `SpMV`, which constitutes a major part of operation time, the rest are `WAXPBY` and `DDOT`.

```
Benchmark Time Summary=
Benchmark Time Summary::Optimization phase=0.578428
Benchmark Time Summary::DDOT=328.875
Benchmark Time Summary::WAXPBY=133.01
Benchmark Time Summary::SpMV=551.773
Benchmark Time Summary::MG=2643.29
Benchmark Time Summary::Total=3657
Floating Point Operations Summary=
Floating Point Operations Summary::Raw DDOT=2.8222e+13
Floating Point Operations Summary::Raw WAXPBY=2.8222e+13
Floating Point Operations Summary::Raw SpMV=2.55977e+14
Floating Point Operations Summary::Raw MG=1.43332e+15
Floating Point Operations Summary::Total=1.74574e+15
Floating Point Operations Summary::Total with convergence overhead=1.61643e+15
GB/s Summary=
GB/s Summary::Raw Read B/W=2940.52
GB/s Summary::Raw Write B/W=679.657
GB/s Summary::Raw Total B/W=3620.18
GB/s Summary::Total with convergence and optimization phase overhead=3317.84
GFLOP/s Summary=
GFLOP/s Summary::Raw DDOT=85.8136
GFLOP/s Summary::Raw WAXPBY=212.179
GFLOP/s Summary::Raw SpMV=463.917
GFLOP/s Summary::Raw MG=542.25
GFLOP/s Summary::Raw Total=477.371
GFLOP/s Summary::Total with convergence overhead=442.01
GFLOP/s Summary::Total with convergence and optimization phase overhead=437.503
```

Fig. 18: Best Performance of HPCG Test

In experiments with a set of input data, we find that the result differs in very small scale, not more than 0.2GFlops. This is also because the local program resulted in a difference, because the vast majority of access is random access memory without Cache acceleration, but the result is very high stability. The specific output files are as **Fig. 18**.

## V. Optimization for AlphaFold3 Inference

### A. Overview

In the ASC25 preliminary, our goal is to optimize the inference speed of AlphaFold3[2] on both GPU and CPU using the data samples provided by the competition organizers. Compared to the baseline on the GPU platform, we achieved a $\mathbf{1.87}\times$ improvement. For CPU inference, we adapted the repository for AlphaFold3 to the CPU platform, using this as our baseline, and then conducted optimization. Compared to the baseline on the CPU platform, we achieved a $\mathbf{4.02}\times$ improvement in total. At the same time, we achieved a maximum acceleration of $\mathbf{30}\times$ improvement on the smallest sample, and other samples also had an acceleration of $\mathbf{3}\times$ to $\mathbf{10}\times$.

a) *Task:*

The objective of this task is to reduce the inference time for the Alphafold3 model, focusing on two main aspects: optimizing the GPU inference process to minimize inference time and migrating the inference codebase from GPU to CPU architecture and optimizing it to reduce the corresponding CPU inference time.

b) *Dataset:*

The committee provided 12 single protein sequence samples and pre-generated MSA and templates.

c) *Model Analysis:*

The structure of the Alphafold3 model is shown in the **Fig. 19**. It can be mainly divided into three module: Evoformer Module (Template Module, MSA Module, Pairformer Module are included), Diffusion Module, Confidence Module.

We use cProfile[3] to analyze program performance and flameprof to generate flamegraph shown in **Fig. 20**. After comparing with the model, we found that the hotspots are concentrated in the functions in **Listing 1**. They correspond to these parts in `Input_embedder`, `Recycle Body`, `Diffusion Module`, and `Confidence Module`, respectively.

### B. Runtime Environment

The hardware resource is shown in **Table VIII** while the software resource is shown in **Table IX**.

TABLE VIII: Hardware Environment

| Platform | Device | Detail |
|---|---|---|
| CPU | CPU | Intel(R) Xeon(R) Platinum 8352V 32-core CPU @ 2.10GHz |
| | GPU | None |
| GPU | CPU | Hygon C86 7380 32-core Processor @ 2.10GHz × 2 |
| | GPU | NVIDIA A800 80GB PCIe × 2 |

TABLE IX: Software Environment

| Platform | Description | Version |
|---|---|---|
| CPU | Operation System | Ubuntu-22.02 |
| | Language | Python-3.11 |
| | Framework | JAX-0.4.34 |
| | | torch-2.5.0+cpu |
| GPU | Operation System | Manjaro-25.0.0 |
| | Language | Python-3.11 |
| | Framework | JAX-0.4.34 |
| | | torch-2.5.1+cu124 |

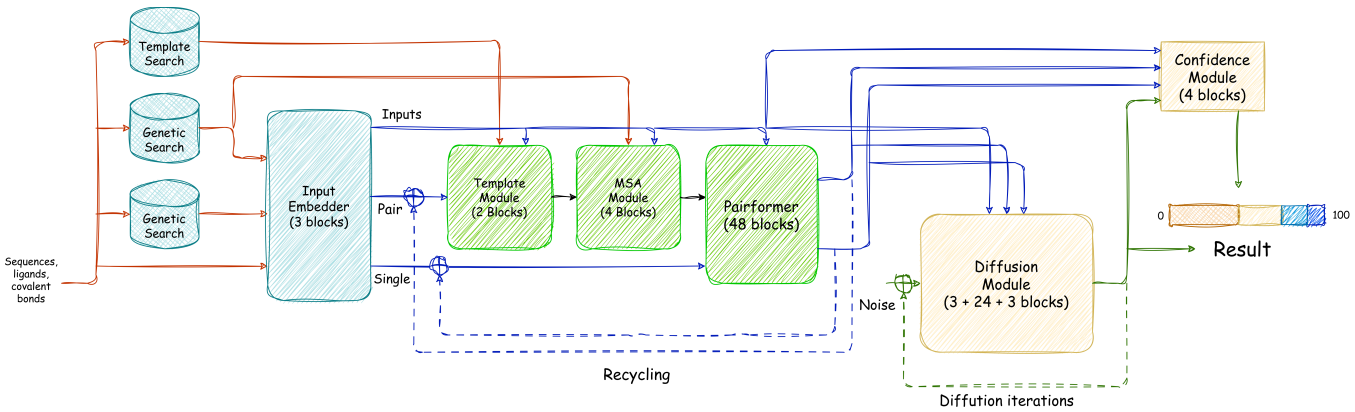



Fig. 19: Structure of Alphafold3

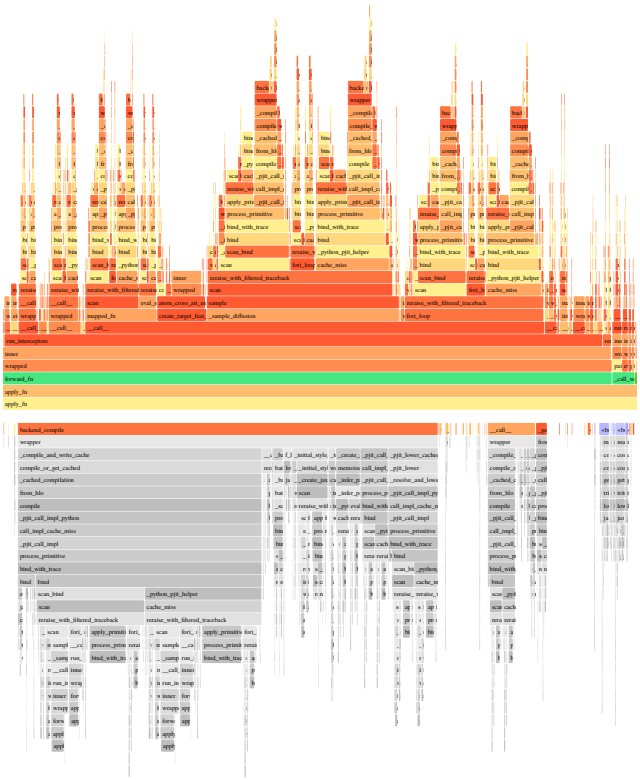


Fig. 20: Flamegraph of `ModelRunner`

```python
"""Create target feature embedding"""
def create_target_feat_embedding(
    batch: feat_batch.Batch,
    config: 'Evoformer.Config',
    global_config: model_config.GlobalConfig,
) -> jnp.ndarray

"""The Loop Body of the function hk.fori_loop"""
def recycle_body(
    _,
    args
) -> embedding, key

"""Denoising Step"""
def _sample_diffusion(
    self,
    batch: feat_batch.Batch,
    embeddings: dict[str, jnp.ndarray],
    *,
    sample_config: diffusion_head.SampleConfig,
) -> dict[str, jnp.ndarray]

@jax.util.wraps(fun, docstr=docstr)
def mapped_fn(
    *args,
    **kwargs
) -> Any
```

Listing 1: The hotspots function of the Model

### C. *Deep Learning Framework Analysis*

JAX[4] has powerful automatic differentiation and TPU compatibility, while Pytorch[5] provides strong scalability. Therefore, we plan to advance optimization work in two directions.

In JAX optimization, we improve the execution efficiency of JAX code through XLA compilation optimization, operator fusion, and reduction of redundant calculations, ensuring that the existing JAX code can achieve maximum performance improvement under the original framework.

For PyTorch migration, we considered PyTorch's advantages in engineering deployment, ecological support, and debugging convenience. We simultaneously migrated the model and explored PyTorch's potential optimization space for reasoning speed so as to provide a better solution when feasible.

Migrating alphafold3 from JAX to Pytorch was necessary to improve optimization efficiency without changing model parameters, and xfold[6] completed the migration and verified that it was consistent with the original implementation.

In order to verify the correctness of the calculation flow, we delved into the xfold source code and finally found the efficiency problem of xfold. We need to denoise five diffusion samples in the Diffusion module. In the source code of alphafold3, Google implemented diffusion sample parallelism through `vmap()`, but in xfold, the five diffusion samples are not parallelized and are generated individually. To solve

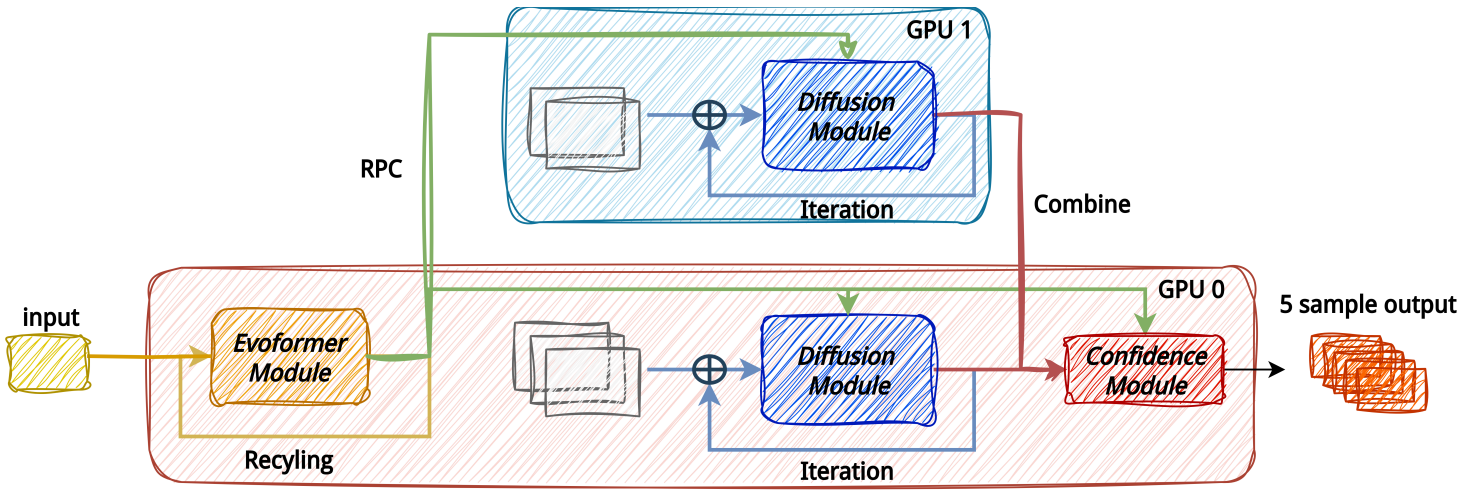



Fig. 21: Data parallel on Diffusion Module. After the Evoformer Module completes its operations on GPU 0, it generates an output batch. This output batch is then transferred to GPU 1 via Remote Procedure Call (RPC). The program subsequently performs the calculations of the Diffusion Module on different GPUs. Once the calculations on these GPUs are completed, the results are combined. Finally, the Confidence Module's calculations are executed on GPU 0

this problem, we implemented diffusion sample parallelism by concatenating the tensors of the five diffusion samples.

### *D. GPU Optimization*

On the GPU platform, we conducted test runs on 12 samples individually using the default settings of AlphaFold3-JAX to establish a baseline.

On **JAX**, we found that because JAX uses JIT, the compilation time for inputs of the same size can be saved when all input files are run together, thus greatly reducing the inference time. At the same time, since JIT builds static graphs, inputs of different sizes need to be recompiled. In the source code, BUCKET is set up to reduce repeated compilation, and the input is padded so that inputs of similar size have the same shape, thereby reducing compilation time and accelerating inference. And by setting environment variables `XLA_FLAGS="--xla_gpu_enable_triton_gemm=false"`, we can solve the known XLA issues that cause a significant increase in compilation time, further speeding up the inference time.

For **Pytorch**, We found that the Diffusion Module used the output batch of the Evoformer Module to calculate five diffusion samples independently. Although the global memory of a single GPU can accommodate the calculation of all diffusion samples, the limited computing cores cannot process five diffusion samples in parallel. Therefore, we can use multiple GPUs to compute different samples in parallel, requiring only a small amount of communication overhead. As is illustrated in the **Fig. 21**, we implemented data parallel on the Diffusion Module via Remote Procedure Call (RPC). This solution has the best performance, and we will present the specific performance improvements later.

### *E. CPU Optimization*

To port Alphafold3-Jax to the CPU, we only need to change the device, disable triton compilation, and use XLA for compilation optimization. This will allow it to run normally. This parameter is mainly located in `run_alphafold.py`

However, only making such modifications in Alphafold3-Jax will cause bug when running on the CPU. An error message `Nan` will be reported in the final stage of extracting results. After our investigation, we found that the problem lies in the `alphafold3/model/diffusion/diffusion_head.py/sample/apply_denoising_step` . As the denoising progresses, it should have gotten 0, but when running on the CPU, it got negative zero, which caused a `Nan` to be generated when performing square root calculation, causing program error. The solution is to add `jnp.abs()`.

```
apply_denoising_step(carry, noise_level):
...
jnp.abs(t_hat**2 - noise_level_prev**2)
...
```

After fixing the bug on the CPU platform, we used it as the baseline. Moreover, our optimization of CPU platforms is based on xfold. The specific optimization plan is as follows:

1) Multiple Thread

To run large models on the CPU, we should use the CPU multi-threaded concurrent execution program to improve inference efficiency. In our experiments, we set the number of threads to 32. We found that CPU utilization was very low when using the original JAX for CPU inference, almost only 800% to 1200%. However, CPU utilization is almost full when we use the torch framework for inference. Moreover, the inference speed has been dramatically improved, so we are implementing the torch framework as our CPU optimization solution.

2) Libiomp and Jemalloc

We use two optimization methods, libiomp and jemalloc, to accelerate the inference speed by 30% to 200% in the implementation of the PyTorch framework.

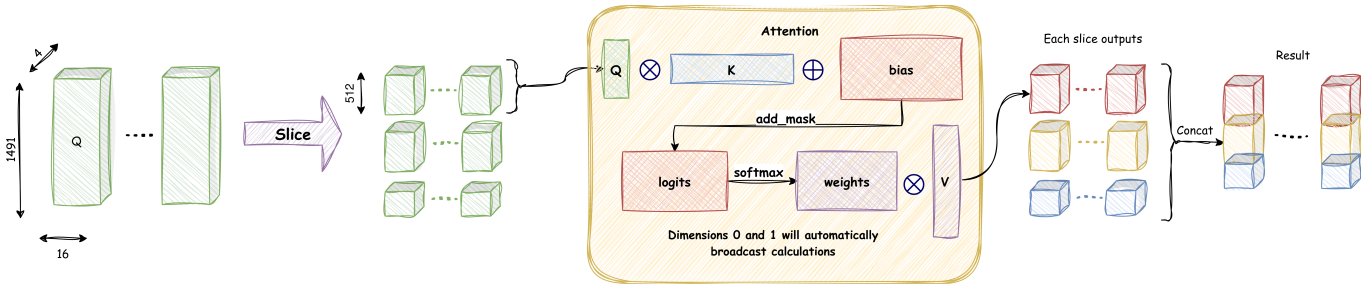



Fig. 22: Slice Attention

OpenMP × Run-time is the core component of the OpenMP API for parallel programming and is responsible for managing OpenMP threads, task scheduling, and synchronization. It allows developers to use multi-core CPUs to improve program parallel computing capabilities. libiomp (Intel OpenMP Library) is an OpenMP runtime library provided by Intel. Its full name is Intel OpenMP Library (libiomp5), libiomp5.so or libiomp5.dll. We use it to speed up inference.

Jemalloc (Jason Evans malloc) is an efficient memory allocation library mainly used to improve memory management performance and reduce memory fragmentation. It is especially suitable for multi-threading and large-scale memory allocation scenarios. We use it instead of the default memory allocation library in the system to maximize CPU utilization.

3) Slice Attention

In the model's Pairformer Module, as the sample increases, the memory usage will increase exponentially, making the inference process extremely prone to memory explosion (OOM).

In order to solve this problem, we perform a slicing operation on the attention in the Pairformer Module of the model (corresponding to the Gridattention in the original alphafold3 code). Significantly reduces memory usage without reducing inference speed.

Specifically, for the input 4-dimensional matrix Q, if its matrix is too large, we divide it according to the second dimension, and at the same time divide the input bias according to the first dimension. The shapes of K and V remain unchanged.

Let's take the sample `1491aa_5KIS` as an example. As shown in the **Fig. 22**, when the second dimension of the matrix is more significant than 1024, we split the second dimension into several 512 matrices. That is, the shape of the original input is (1491, 4, 1491, 16), and the shape after segmentation is (1491, 4, 512, 16). Then we loop to do attention on each sliced matrix input. After the output is obtained, the results are spliced together to obtain the final result.

During our experiments, we found that when `slice_attention` is not used, the memory usage of the `1491aa_5KIS` sample can reach up to 130GiB. However, after using `slice_attention`, the maximum memory required is only 70GiB, which is almost reduced by half and hardly reduces the inference speed. Moreover, since the calculation results are the same, there will be no loss of accuracy. Finally, the memory usage can be even smaller by dividing the matrix into smaller pieces than 512.

### *F. Results Reproduction and Analysis*

a) *Reproduction:*

For specific steps, please refer to the **README** in the project file.

b) *Analysis:*

After a series of optimizations, the inference speed of Alphafold3 has been significantly improved. We conducted experimental tests on the GPU platform and the CPU platform. Compared to the baseline on the GPU platform, we achieved a $\mathbf{1.87}\times$ improvement. Compared to the baseline on the CPU platform, we achieved a $\mathbf{4.02}\times$ improvement in total. At the same time, we achieved a maximum acceleration of $\mathbf{30}\times$ improvement on `37aa_3JO9`, and other samples also had an acceleration of $\mathbf{3}\times$ to $\mathbf{10}\times$. The experimental results are summarized in **Table X**, **Fig. 23** and **Table XI**, **Fig. 24** respectively.

In addition to improving the inference speed, we also strictly evaluated the confidence metrics of the model output to ensure that the optimization does not affect the accuracy of the prediction results. The comparison of important confidence metrics is shown in **Fig. 25**. Through comparative analysis of the CPU and GPU platforms, we found that our optimization solution accelerated the inference speed while maintaining the same confidence metrics as the baseline, indicating that the optimized model can still ensure the reliability and biological plausibility of the prediction.

The visualization results of protein prediction for the 12 samples can be found in the Appendix A.

### *G. Future Work*

a) *AsyncDiff:*

In AlphaFold3's Diffusion Module, the denoise step has a strong time dependency due to its iterative nature, limiting parallelism. AsyncDiff[7] accelerates diffusion by enabling model parallelism across multiple devices. It splits the noise

prediction model into components and assigns each to a different device. By leveraging the similarity between hidden states in consecutive steps, AsyncDiff transforms sequential denoising into an asynchronous process, allowing parallel computation and improving efficiency.

b) *Distribute System:*

Due to limited device availability, we optimized the calculation on the GPU during the Diffusion phase by utilizing two GPUs to handle computation for different diffusion samples. Furthermore, leveraging more GPUs for computing various samples is a potential future direction, which will require careful consideration of the trade-offs between communication and computation.

In the CPU's Diffusion phase, after completing the calculation for the Evoformer, the output batch can be broadcast to multiple nodes. This allows for the use of multi-node computing for different diffusion samples, similar to our optimization approach on the GPU.

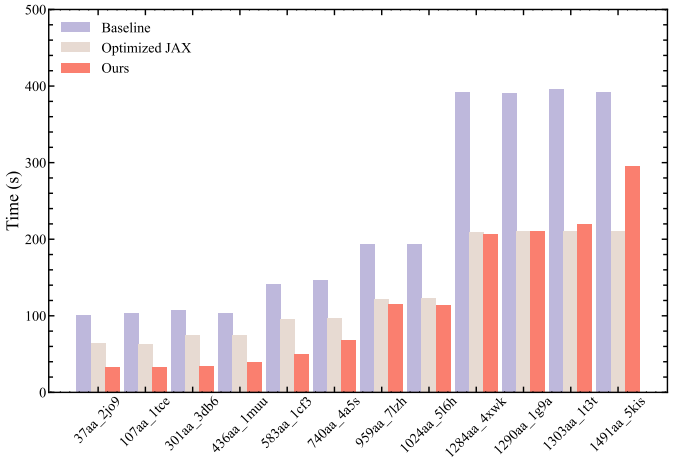



Fig. 23: GPU

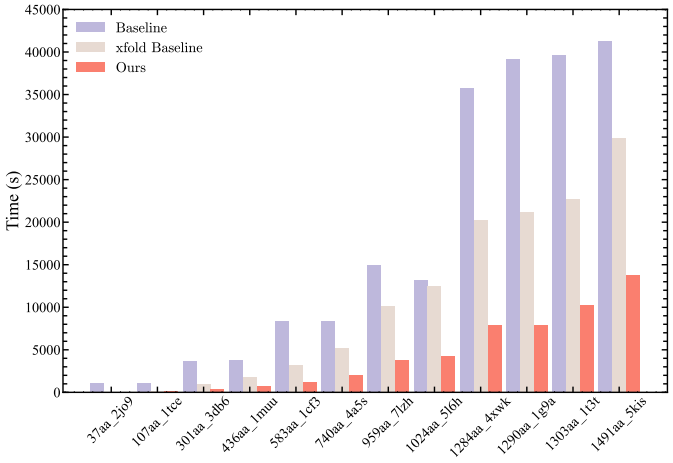



Fig. 24: CPU

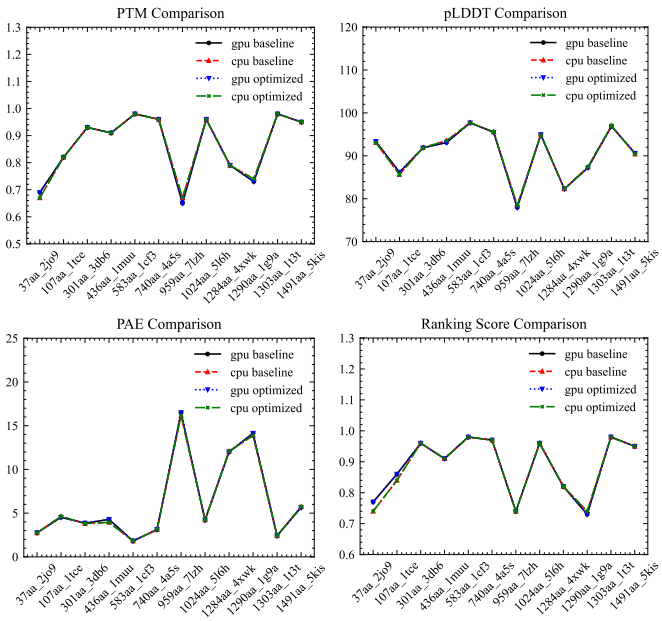



Fig. 25: Comparison of Results

TABLE X: GPU Platform Results

| Sample | Baseline/s | Optimized-JAX/s | Ours/s |
|---|---|---|---|
| 37aa_2JO9 | 101.51 | 63.93 | 33.22 |
| 107aa_1TCE | 103.29 | 62.65 | 33.40 |
| 301aa_3DB6 | 107.70 | 74.50 | 34.25 |
| 436aa_1MUU | 104.13 | 74.90 | 38.92 |
| 583aa_1CF3 | 141.41 | 95.66 | 49.39 |
| 740aa_4A5S | 146.77 | 96.71 | 68.33 |
| 959aa_7LZH | 193.65 | 121.41 | 114.99 |
| 1024aa_5L6H | 193.05 | 122.53 | 113.91 |
| 1284aa_4XWK | 391.78 | 209.62 | 206.84 |
| 1290aa_1G9A | 391.32 | 210.38 | 211.21 |
| 1303aa_1T3T | 396.05 | 209.99 | 220.18 |
| 1491aa_5KIS | 392.41 | 210.38 | 295.06 |
| Total Time | 2663.07 | 1552.66 | **1419.70** |

TABLE XI: CPU Platform Results

| Sample | Baseline/s | xfold-Baseline/s | Ours/s |
|---|---|---|---|
| 37aa_2JO9 | 1099.73 | 47.27 | 33.72 |
| 107aa_1TCE | 1097.56 | 114.62 | 95.58 |
| 301aa_3DB6 | 3664.60 | 1007.84 | 383.94 |
| 436aa_1MUU | 3802.55 | 1794.29 | 714.56 |
| 583aa_1CF3 | 8384.13 | 3230.05 | 1231.96 |
| 740aa_4A5S | 8335.49 | 5227.67 | 2079.65 |
| 959aa_7LZH | 14903.64 | 10115.15 | 3756.03 |
| 1024aa_5L6H | 13140.91 | 12516.74 | 4256.91 |
| 1284aa_4XWK | 35774.90 | 20223.38 | 7855.95 |
| 1290aa_1G9A | 39209.44 | 21157.53 | 7856.85 |
| 1303aa_1T3T | 39656.09 | 22665.59 | 10274.63 |
| 1491aa_5KIS | 41220.65 | 29889.52 | 13786.94 |
| Total Time | 210289.69 | 127989.65 | **52326.72** |

TABLE XII: BASELINE OF THE WORKFLOW

| Step | Configuration | Sample | Time Cost (Seconds) |
|---|---|---|---|
| cutseq | 32 threads | SRR23538290 | 173 |
| | | SRR23538291 | 237 |
| | | SRR23538292 | 222 |
| hisat-3n index<br>samtools fastq, view, sort | 32 threads | SRR23538290 | 998 |
| | | SRR23538291 | 1495 |
| | | SRR23538292 | 1516 |
| umicollapse | 32 threads | SRR23538290 | 1115 |
| | | SRR23538291 | 1534 |
| | | SRR23538292 | 1402 |
| samtools view, index | 32 threads | SRR23538290 | 123 |
| | | SRR23538291 | 171 |
| | | SRR23538292 | 157 |
| samtools view<br>hisat-3n-table<br>cut and gzip | 16 threads -m unfiltered | SRR23538290 | 2617 |
| | 16 threads -m filtered | | 2680 |
| | 16 threads -u unfiltered | | 3026 |
| | 16 threads -u filtered | | 3142 |
| | 16 threads -m unfiltered | SRR23538291 | 3458 |
| | 16 threads -m filtered | | 3841 |
| | 16 threads -u unfiltered | | 3854 |
| | 16 threads -u filtered | | 3467 |
| | 16 threads -m unfiltered | SRR23538292 | 3114 |
| | 16 threads -m filtered | | 3553 |
| | 16 threads -u unfiltered | | 3597 |
| | 16 threads -u filtered | | 2655 |

## VI. RNA m$^5$C MODIFICATION SITE DETECTION AND PERFORMANCE OPTIMIZATION CHALLENGE

### A. Overview

In this challenge, we analyzed the workflow for detecting RNA m5C modifications and conducted a hotspot analysis. For the bottleneck program `hisat-3n-table`, we examined the software's program structure and control flow and proposed an optimization solution. The solution achieved an average speedup of $\mathbf{2.6}\times$ by rewriting inefficient data structures and control flows. Then, we restructured and rewrote the entire program, achieving an average speedup of $\mathbf{6.3}\times$ on the test samples. We also implemented parallelization of the whole workflow using SSH.

### B. Baseline and Bottleneck

We ran the benchmark program, and the time distribution of each step is shown in the **Table XII**.

It is evident that the bottleneck in the process lies in the `hisat-3n-table` program, which accounts for xxx% of the total runtime.

Note that in the `hisat-3n-table`, using more than 16 threads actually reduces performance, which suggests that the implementation of `hisat-3n-table` has limitations in terms of parallelism. So, we used its best performance as the baseline.

Therefore, we will focus on performance analysis and optimization of this program to improve the overall efficiency of the process.

### C. Anlysis

a) *Task:*

`hisat-3n-table` can be used to generate a 3N-conversion-table from a sorted SAM file generated by `hisat-3n`[8].

The main function of `hisat-3n-table`, which is shown in **Fig. 26**, is to read the preprocessed sample data (such as sorted and deduplicated data) in units of Alignment, align it with the reference positions loaded from the reference genome file, store the processed results in the corresponding position, and finally produce the output.

b) *Code Structure:*

The original program, whose structure is shown clearly in **Fig. 27**, consists of a reading thread, multiple worker threads, and an output thread, using two thread-safe data queues for communication between the reading thread and the worker

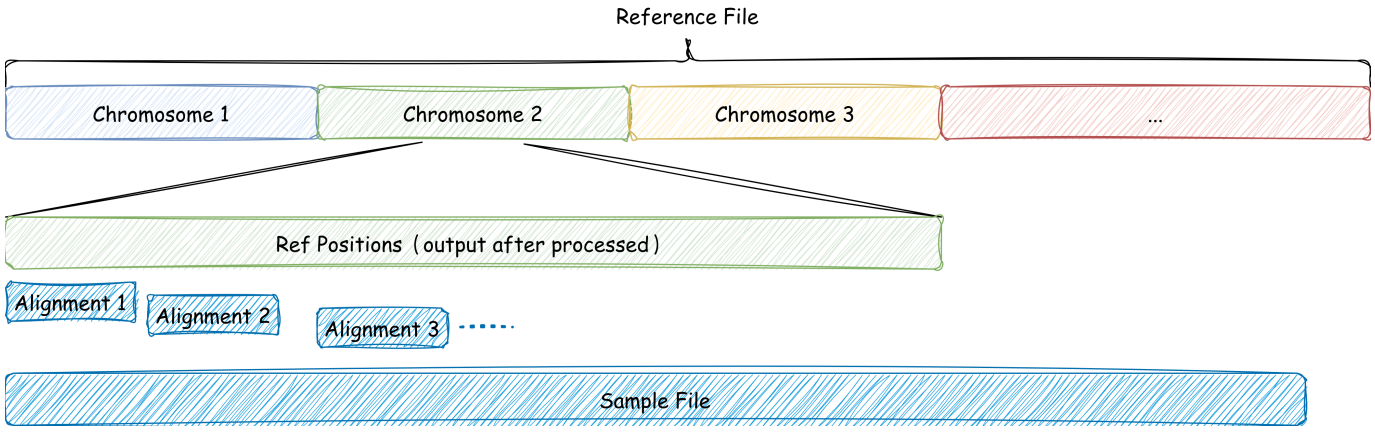



Fig. 26: Function of hisat-3n-table

threads, as well as between the worker threads and the output thread.

The reading thread is responsible for reading the reference chromosome file and the sample data file. It continuously filters and reads the alignment lines from the sample data and places them into the queue. If a newly read line exceeds the currently loaded reference position, it must wait for the worker threads to process the current alignment in the queue, then place the current reference positions block into the output queue, load a new reference position, and then place it into the work queue.

The worker threads parse the read alignment lines into alignment objects and match them with the currently loaded reference position, writing each matching result directly into the corresponding reference position.

The output thread is responsible for outputting each position from the reference position blocks in the output queue directly to a specified file.

To control memory usage and reduce the number of memory allocation requests, the original program employs two pools (a line pool and a position pool) to manage string objects and position objects.

During data reading, strings and position information are read from the file into memory, and free line and position objects are retrieved from the pools to serve as containers for data processing. Once the data processing is complete, the line and position objects are returned to the pools for reuse in future operations.

c) *Hotspot:*

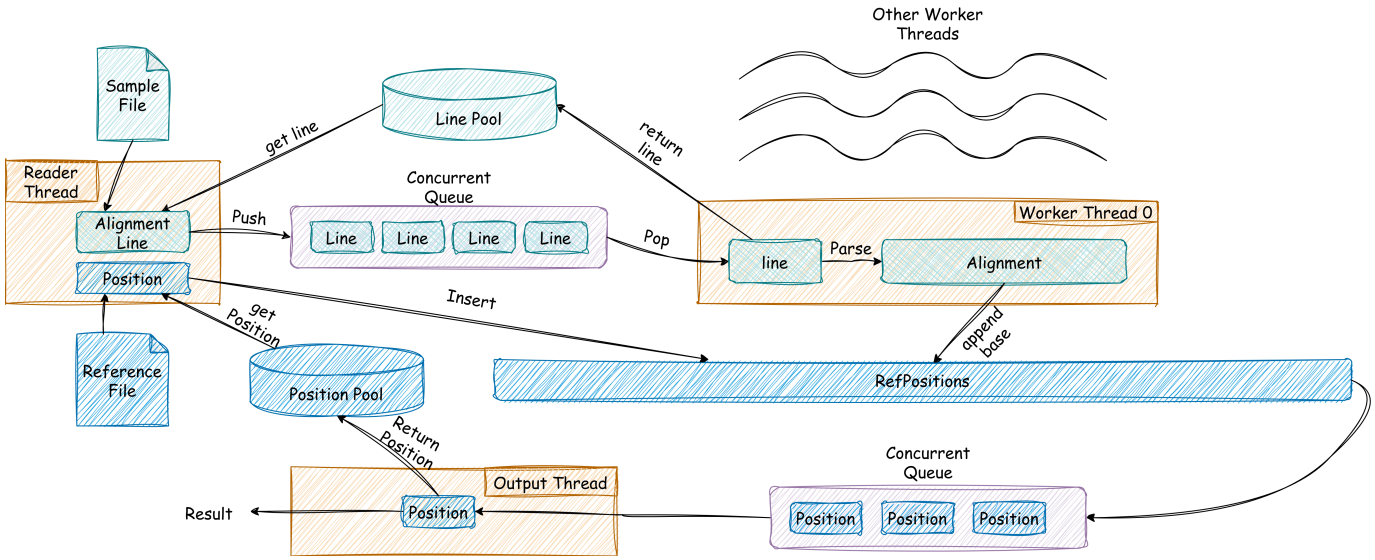



Fig. 27: Original Code Structure

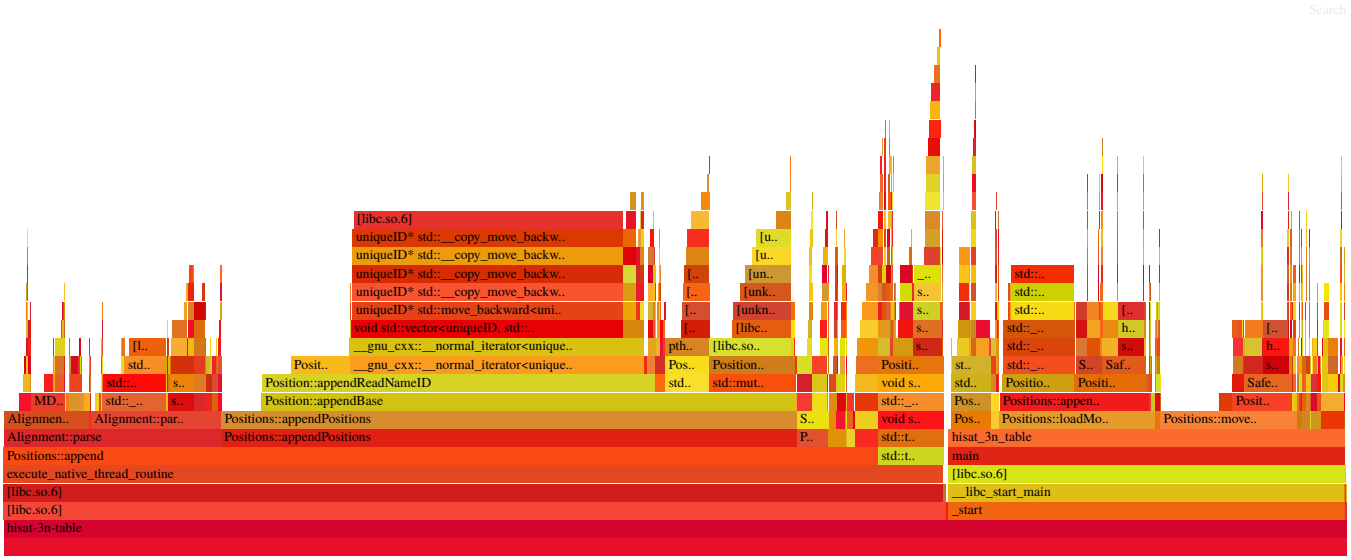



Fig. 28: Flamegraph of original `hisat-3n-table`

We used the `perf`[9] tool to perform a performance analysis of `hisat-3n-table` and generated a flame graph, as shown in **Fig. 28**.

Based on the above analysis, we summarize the primary sources of performance bottlenecks in the `hisat-3n-table`.

- Low efficiency in accessing and loading the reference file; the Reader thread must read the entire file in chunks.
- Low concurrency in the program's input and output; the reading of the Sample file and Reference is handled by only one thread, limiting the performance of the worker threads.
- Inefficient structural design of the program; on one hand, the reader and worker threads must wait for each other, leading to time consumption; on the other hand, the complex concurrency control logic reduces the program's maintainability.
- Inefficient implementation of the program's key data structure, SafeQueue, which heavily relies on mutexes and sleeps for concurrency control.

To address these issues, we propose a series of optimization strategies to improve the program's performance.

d) *A potential bug:*

We found that in `hisat-3n-table`, the original program sorts each reference chromosome by name and assumes that the input sample data is also sorted by the names of the reference chromosomes. This allows the original program to load the reference file in a unidirectional manner, which means it does not need to reload chromosomes that have already been processed.

We will talk about it in detail at **Section VI.E**

### *D. Using efficient data structure*

We have noticed that the SafeQueue used in the source program relies heavily on mutex locks to handle concurrency control, which is undoubtedly inefficient and cumbersome. In response, a more efficient and concurrency-safe data structure can be used as a replacement, which would improve the program's performance without disrupting the original program structure and execution flow. After repeated experimentation and comparison, our final solution is to replace it with a lock-free queue data structure. Specifically, we used the lockfree::queue from the open-source Boost library. Additionally, because no modifications are made to the execution flow of the source program, we can ensure the program is correct and accurate.

a) *Lock-Free Queue:*

A lock-free queue is a type of data structure that allows multiple threads to access and modify the queue concurrently without the need for locks[10]. In traditional queue implementations, locks are used to ensure that only one thread can modify the queue at a time, which can lead to performance bottlenecks, especially in highly concurrent systems. In contrast, lock-free queues allow for safe and efficient access by multiple threads simultaneously, as they rely on atomic operations such as compare-and-swap (CAS) to manage concurrency.

The main advantage of lock-free queues is that they eliminate the overhead and potential deadlocks associated with traditional locking mechanisms. This makes them particularly useful in high-performance applications where minimizing latency and maximizing throughput are critical. A lock-free

queue ensures that at least one thread will always make progress in the system, even if other threads are blocked or delayed.

b) *Boost Library and Lock-Free:*

The Boost library is a comprehensive collection of peer-reviewed, portable, and high-quality C++ libraries[11]. It provides support for various functionalities, including containers, algorithms, threading, and more. One of the notable components of Boost is its lock-free module, which offers implementations of lock-free data structures, including lock-free queues.

Boost's `lockfree::queue` is a widely used implementation of a lock-free queue. It uses atomic operations to ensure that multiple threads can safely push and pop items from the queue without the need for mutexes. This queue is particularly useful in multi-threaded environments where performance is crucial, and it ensures that threads can work concurrently without blocking each other, improving overall system efficiency.

The `lockfree::queue` in Boost is designed to work seamlessly in both single-producer and multi-consumer scenarios. It uses a memory model that ensures thread safety and eliminates race conditions while maintaining high performance under heavy concurrency. By utilizing this lock-free queue, we can achieve better throughput and lower latency in our program without the inefficiencies introduced by traditional locking mechanisms.

### *E. Redesign the program structure*

In the solution, we have decided to redesign the program structure. As shown in **Fig. 29**, we have restructured the program.

We abstract the reference file into a unified address space, where each position can be accessed through the address tuple:

$$(\text{chromosome name}, \text{offset}) \tag{4}$$

For accessing this address space, we draw inspiration from database[12] programs and abstract a buffer pool on top of it to manage the loading and the output of reference positions. The buffer pool provides a concurrent-safe `deref` interface for worker threads, which read addresses in the aforementioned format and return a position's reference.

Each worker thread can read alignments from the sample file in parallel and use the `deref` interface to obtain the corresponding position for appending operations. Notably, the `deref` operation will select or wait for free blocks to load when the target position is not loaded, it will also output the existing data. The loading and output operations are handled

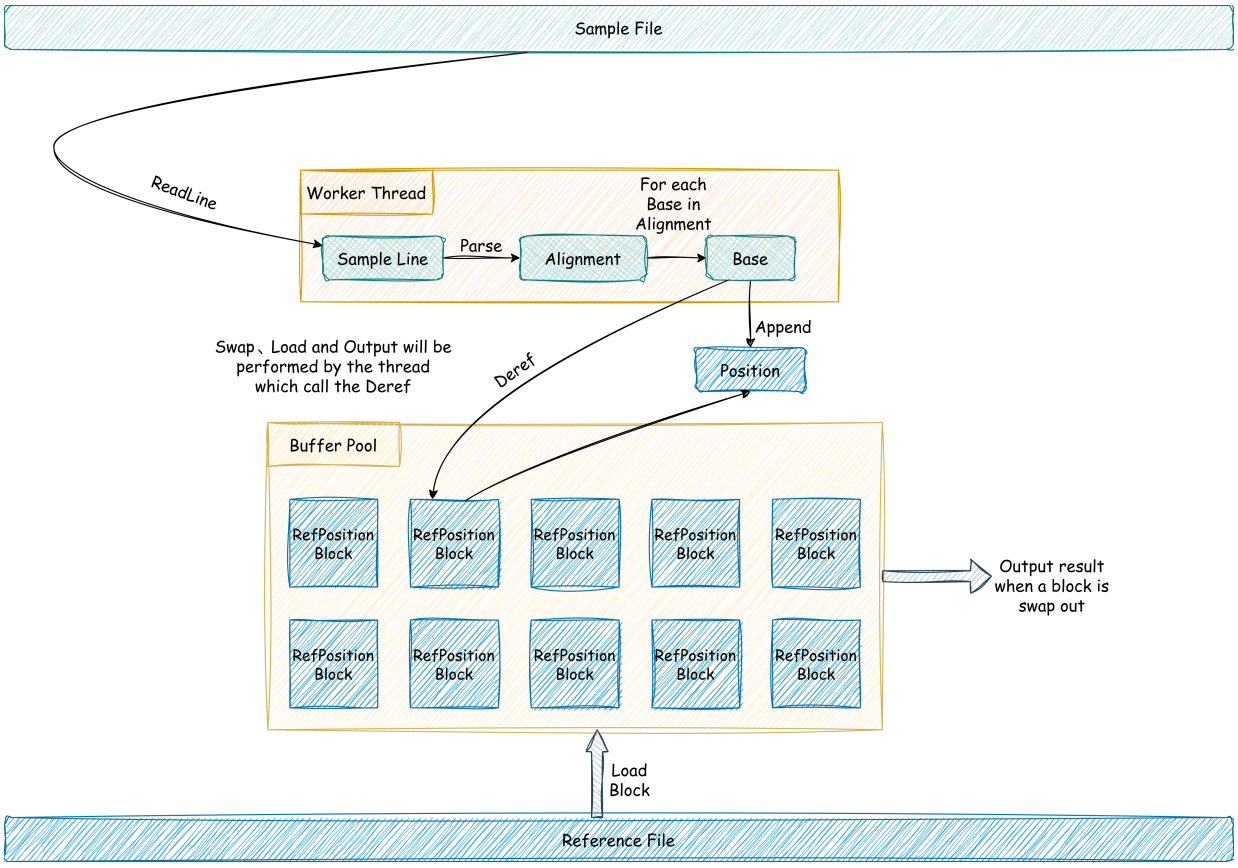



Fig. 29: Optimized program structure

by the thread that calls `deref`, enabling both the loading and output of the reference file to be parallelized.

Furthermore, this design completely confines the logic of concurrency control within the buffer pool, simplifying the logic of worker threads and enhancing the maintainability of the program.

There are some details of this solution below.

a) *Preprocess the Reference File:*

In the original reference file, there are numerous newline characters interspersed among the base sequences of the same chromosome, which complicates reading and loading. It prevents random access to the desired positions, requiring that all previous sequences be loaded first and then a counter must be used to determine whether the specific position has been reached.

To address this issue, we propose a preprocessing solution: at the startup of the program, we will preprocess the reference file by merging the base sequences of the same chromosome into a single line. This way, we can directly read the desired position by calculating it without having to sequentially read the entire file.

b) *Efficient implementation of the buffer pool:*

The buffer pool is a key data structure in the rewritten program, providing an abstraction of the reference file for worker threads and offering a concurrency-safe interface. To enhance the performance of the buffer pool, we have implemented the following design.

Each reference position block in the buffer pool is defined as shown in **Listing 2**.

```
struct buffer_t {
  std::shared_ptr<std::string> name;
  std::vector<position_t> data;
  std::atomic<uint32_t> ref_cnt{0};
  int id;
  uint64_t begin;
  uint64_t end;
  bool dirty_bit{false};
}
```

Listing 2: definition of reference position block

In this design, `ref_cnt` is used to track the reference count of each block, while `dirty_bit` indicates whether the block should be output. In the buffer pool, we utilize a `vector` to store all the blocks.

To avoid frequent operations on the reference count and comparisons of chromosome name strings, we have introduced a `pin` interface for the buffer pool. This interface returns a `buffer_context_t` object, which provides the `deref` interface. This object maintains the reference count of the block being used, modifying it during construction and destruction. The definitions of the `pin` interface and `buffer_context_t` are shown in **Listing 3**.

```
struct buffer_context_t {
  pool_t* pool;
  std::shared_ptr<std::string> name;
  int buffer_id{-1};
  size_t cur_begin{0};
  size_t cur_end{0};
};

auto buffer_pool_t::pin(
  std::shared_ptr<std::string> name
) -> std::unique_ptr<buffer_handler_t>;

auto buffer_context_t::deref(
  uint64_t addr
) -> position_t*
```

Listing 3: definition of pin and deref

The buffer pool, during its construction, requires a callback function `onswap`, which will be called when a block is replaced. This function is used to output the processed block.

To ensure the correctness of the program's output and to take advantage of the sequential access characteristic of the reference file as much as possible, we designed a specialized replacement strategy for this program. This strategy directly selects the block that is sorted last in the reference file for replacement, ensuring that the replaced block will not be referenced again. In the specific implementation, we used a binary heap to maintain the blocks to be replaced, aiming to maximize efficiency.

c) *Reduce memory allocation:*

For fixed strings in the program, such as chromosome names, the original implementation used a copying approach, resulting in numerous memory allocation and deallocation operations. To reduce the frequency of memory allocations, we changed the handling of these strings to store pointers instead, thereby avoiding the overhead of memory allocation and release. To ensure memory safety, we manage these strings using `shared_ptr`, which helps prevent memory leaks and other related issues.

### *F. Parallelization of the entire workflow*

As of m5C-UBSseq, workflows serve three separate processes so that we can use the most convenient multi-machine parallel strategy in these three independent processes. We designed a makefile file to control the different stages of the workflow, and for each stage, we used ssh to specify the corresponding server to run the instructions. In the cluster setup, we used nfs-server to share a folder, using the same user name and folder path to ensure the regular operation of multiple machines. In addition, we use run.sh to make it easier to design the entire workflow from a more user-oriented perspective, which is detailed in the **README** in the usage file.

### G. Results & Analysis

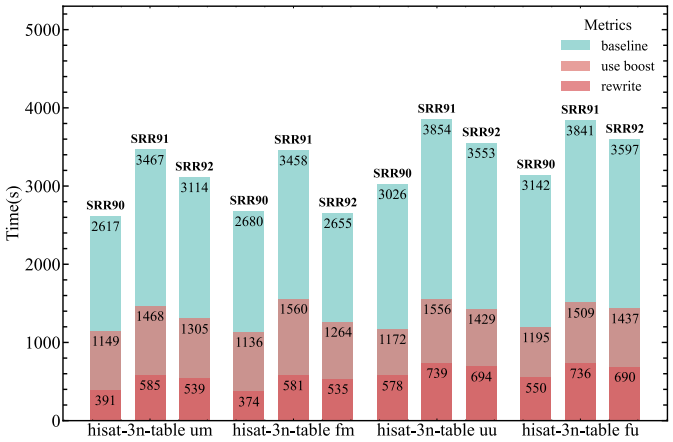



Fig. 30: Program execution time (single-thread)

TABLE XIII: SPEEDUP IN DIFFERENT WORKLOAD

| Sample | Size | Baseline(s) | Optimized(s) |
|---|---|---|---|
| SRR72 | 159 MB | 72 | 2 |
| SRR79 | 4 GB | 331 | 3 |
| SRR90 | 40 GB | 2617 | 389 |

a) *Speedup:*

We compared the execution time of the optimized program with the original program under a single-threaded scenario using the test data provided in the competition, as shown in **Fig. 30**. The rewritten program significantly improved efficiency.

Additionally, we conducted further testing on different workloads, as shown in the **Table XIII**. For data with lower density, the rewritten program achieved up to a 10 × speedup. For data with higher density, it provided an average speedup of around 6 ×.

Under workloads with dense data, our tests revealed that the speedup primarily comes from a more reasonable program structure and an efficient buffer pool implementation. The buffer pool enhances the granularity of loading reduces the loading of invalid data and maximizes the usage of loaded data, thereby minimizing thread wait times.

In sparser datasets, the speedup also stems from the preprocessing of the reference file. The preprocessed reference file supports random access, allowing us to skip unnecessary data segments, resulting in higher load efficiency.

b) *Parallel efficiency:*

We also tested the parallel efficiency of the program. We plotted the performance of both the original and optimized programs with different thread counts on the same dataset in **Fig. 31**.

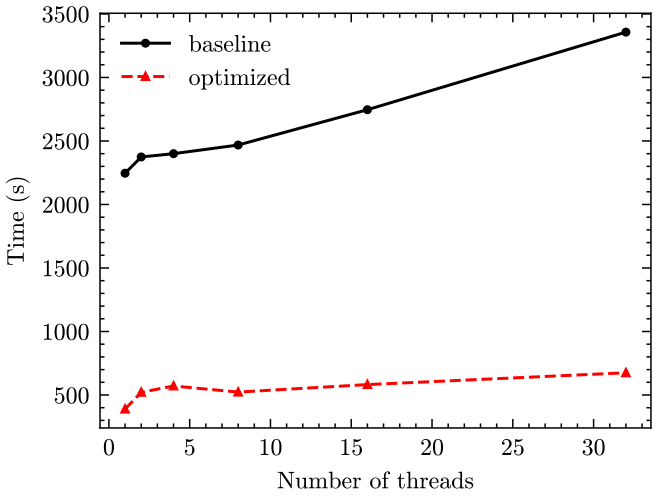



Fig. 31: Program execution time (single-thread)

Both the rewritten program and the original program exhibited low parallel efficiency, with execution times increasing as the number of threads increased. Our analysis revealed that this issue was caused by inappropriate parallel loading due to the characteristics of the program itself.

Currently, the solution to this problem is to split the input sample files and process them using multiple processes before merging the results, which can lead to higher efficiency.

c) *Buffer efficiency:*

We tested the efficiency of the buffer pool. Under various workloads, the buffer pool provided excellent caching performance, reducing thread wait times and overhead.

In single-threaded mode, we conducted buffer hit rate tests using the program provided in the competition, with results shown in the **Table XIV**.

TABLE XIV: BUFFER HIT RATIO

| Sample | Size | Deref Times | Miss Times | Hit Rate |
|---|---|---|---|---|
| SRR72 | 159 MB | 55 | 16 | 70.9% |
| SRR79 | 4 GB | 3212 | 418 | 86.9% |
| SRR90 | 40 GB | 237736697 | 1982068 | 99.1% |

In tests with different data, the buffer pool's dereferencing consistently achieved over 70% hit rates. This success is attributed to our chromosome position-based swap strategy, which leverages program logic to retain blocks that are likely to be used again.

### H. Conclusion

After optimizing the entire workflow for a single `hista-3n-table` program, we can stably reach a speedup to 6 × ∼ 10 ×; in the dimension of the whole workflow, we use multi-machine parallel and can achieve 3 × the optimization rate; in four independent hisat-3n-table workflows, we

can have an speedup to 7 $\times$. Overall, the original workflow took 14h, and now we can complete it in less than 2h

### *I. Proposal for further improvements*

For the following work, we plan to optimize the multi-threaded parallelism of our program again so that it can take advantage of multithreading and achieve better speed results.

## References


[1] "HPL - A Portable Implementation of the High-Performance Linpack Benchmark for Distributed-Memory Computers." Accessed: Jan. 17, 2025. [Online]. Available: https://www.netlib.org/benchmark/hpl/

[2] J. Abramson *et al.*, "Accurate structure prediction of biomolecular interactions with AlphaFold 3," *Nature*, vol. 630, no. 8016, pp. 493–500, 2024, doi: 10.1038/s41586-024-07487-w.

[3] "python/cpython at 3.13." [Online]. Available: https://github.com/python/cpython

[4] J. Bradbury *et al.*, "JAX: composable transformations of Python+NumPy programs." [Online]. Available: http://github.com/jax-ml/jax

[5] J. Ansel *et al.*, "PyTorch 2: Faster Machine Learning Through Dynamic Python Bytecode Transformation and Graph Compilation." [Online]. Available: https://pytorch.org/assets/pytorch2-2.pdf

[6] "Shenggan/xfold: Democratizing AlphaFold3: an PyTorch reimplementation to accelerate protein structure prediction." [Online]. Available: https://github.com/Shenggan/xfold

[7] Z. Chen, X. Ma, G. Fang, Z. Tan, and X. Wang, "AsyncDiff: Parallelizing Diffusion Models by Asynchronous Denoising," *arXiv preprint arXiv:2406.06911*, 2024.

[8] "Graph-based alignment of next generation sequencing reads to a population of genomes." Accessed: 2021. [Online]. Available: https://daehwankimlab.github.io/hisat2/hisat-3n/

[9] "Perf: Linux Profiling with Performance Counters ." Accessed: Dec. 01, 2024. [Online]. Available: https://perfwiki.github.io/main/

[10] "Non-blocking algorithms." [Online]. Available: https://en.wikipedia.org/wiki/Non-blocking_algorithm#Lock-freedom

[11] "Boost C++ Libraries." [Online]. Available: https://www.boost.org/

[12] H. F. K. Avi Silberschatz and S. Sudarshan, *Database System Concepts*. McGraw-Hill Inc.